

Лекция 24. Advanced WorkManager и тестирование

В уроке Background Work with WorkManager вы узнали, как выполнять работу в фоновом режиме (не в главном потоке) с помощью WorkManager. В этом кодебалете вы продолжите изучение функциональности WorkManager для обеспечения уникальности работы, маркировки работы, отмены работы и ограничений работы. В конце урока вы узнаете, как писать автоматизированные тесты для проверки правильности работы рабочих и возврата ожидаемых результатов. Вы также узнаете, как использовать инспектор фоновых задач, предоставляемый Android Studio, для проверки очереди рабочих.

Что вы будете создавать

В этом уроке вы обеспечите уникальность работы, пометку работы, отмену работы и реализацию ограничений на работу. Затем вы узнаете, как написать автоматизированные UI-тесты для приложения **Blur-O-Matic**, которые проверят функциональность трех рабочих, созданных в уроке Background Work with WorkManager:

- BlurWorker
- CleanupWorker
- SaveImageToFileWorker

Что вы узнаете

- Обеспечение уникальности работ.
- Как отменить работу.
- Как определять ограничения на работу.
- Как писать автоматизированные тесты для проверки функциональности рабочих.
- Основы проверки рабочих в очереди с помощью инспектора фоновых задач.

Что вам понадобится

- Последняя стабильная версия Android Studio
- Завершение урока Background Work with WorkManager Устройство или эмулятор Android

Подготовка к работе

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-workmanager.git
$ cd basic-android-kotlin-compose-training-workmanager
$ git checkout intermediate
```

Обеспечение уникальности работы

Теперь, когда вы знаете, как создавать цепочки рабочих, пришло время разобраться с другой мощной функцией WorkManager: уникальными последовательностями работ.

Иногда вам нужно, чтобы одновременно выполнялась только одна цепочка работ. Например, у вас есть цепочка работ, которая синхронизирует ваши локальные данные с сервером. Вероятно, вы хотите, чтобы первая синхронизация данных завершилась, прежде чем начинать новую. Для этого вы используете `beginUniqueWork()` вместо `beginWith()` и указываете уникальное строковое имя. Это имя присваивается всей цепочке рабочих запросов, чтобы вы могли ссылаться на них и запрашивать их вместе.

Также необходимо передать объект `ExistingWorkPolicy`. Этот объект указывает ОС Android, что произойдет, если работа уже существует. Возможные значения `ExistingWorkPolicy` - `REPLACE`, `KEEP`, `APPEND` или `APPEND_OR_REPLACE`.

В этом приложении вы хотите использовать `REPLACE`, потому что если пользователь решит размыть другое изображение до того, как закончится текущее, вы хотите остановить текущее и начать размывать новое изображение.

Вы также хотите убедиться, что если пользователь нажмет кнопку «Начать», когда рабочий запрос уже находится в очереди, то приложение заменит предыдущий рабочий запрос на новый. Нет смысла продолжать работу над предыдущим запросом, поскольку приложение все равно заменит его новым.

В файле `data/WorkManagerBluromaticRepository.kt` в методе `applyBlur()` выполните следующие действия:

Удалите вызов функции `beginWith()` и добавьте вызов функции `beginUniqueWork()`. В качестве первого параметра функции `beginUniqueWork()` передайте константу `IMAGE_MANIPULATION_WORK_NAME`. Для второго параметра, параметра `existingWorkPolicy`, передайте `ExistingWorkPolicy.REPLACE`. Для третьего параметра создайте новый `OneTimeWorkRequest` для `CleanupWorker`.

```
data/WorkManagerBluromaticRepository.kt ``kt import androidx.work.ExistingWorkPolicy import
com.example.blumatic.IMAGE_MANIPULATION_WORK_NAME ... // ЗАМЕНИТЕ ЭТОТ КОД: // var
continuation = workManager.beginWith(OneTimeWorkRequest.from(CleanupWorker::class.java)) // WITH var
continuation = workManager .beginUniqueWork( IMAGE_MANIPULATION_WORK_NAME,
ExistingWorkPolicy.REPLACE, OneTimeWorkRequest.from(CleanupWorker::class.java) ) ...
```

`Blur-O-Matic` теперь размывает только одно изображение за раз.

4. Метки и обновление пользовательского интерфейса в зависимости от статуса работы

Следующее изменение, которое вы сделаете, - это то, что приложение показывает, когда работа выполняется. Информация, полученная о выполнении работ, определяет, как должен измениться пользовательский интерфейс.

В этой таблице показаны три различных метода, которые можно вызвать для получения информации о работе:

Type	WorkManager	Method	Description
:---	:-----	:-----	

|Получение информации о работе по идентификатору|getWorkInfoByIdLiveData()|Эта функция возвращает один LiveData<WorkInfo> для конкретного WorkRequest по его идентификатору.|

|Получение работ по уникальному имени цепочки|getWorkInfosForUniqueWorkLiveData()|Эта функция возвращает LiveData<List<WorkInfo>> для всех работ в уникальной цепочке WorkRequests.|

|Получение работ по тегу|getWorkInfosByTagLiveData()|Эта функция возвращает LiveData<List<WorkInfo>> для тега.|

> Примечание: WorkManager предоставляет некоторые API в виде LiveData. Мы используем API LiveData, но конвертируем их и используем как поток. Если в будущем API изменятся, мы сможем соответствующим образом обновить codelab.

> Объект WorkInfo содержит сведения о текущем состоянии запроса на выполнение работы, включая:

Заблокирована ли работа, отменена, завершена, провалена, запущена или успешно завершена.

Если WorkRequest завершен, и любые выходные данные, полученные в результате работы.

Эти методы возвращают LiveData. LiveData - это наблюдаемый держатель данных с учетом жизненного цикла. Мы преобразуем его в поток объектов WorkInfo, вызвав .asFlow().

Поскольку вас интересует, когда сохраняется конечное изображение, вы добавляете тег в рабочий запрос SaveImageToFileWorker, чтобы можно было получить его WorkInfo из метода getWorkInfosByTagLiveData().

Другой вариант - использовать метод getWorkInfosForUniqueWorkLiveData(), который возвращает информацию обо всех трех WorkRequests (CleanupWorker, BlurWorker и SaveImageToFileWorker). Недостатком этого метода является то, что вам потребуется дополнительный код, чтобы найти необходимую информацию о SaveImageToFileWorker.

Пометить запрос на выполнение работы

Пометка работы выполняется в файле data/WorkManagerBluromaticRepository.kt внутри функции applyBlur().

При создании рабочего запроса SaveImageToFileWorker пометьте работу, вызвав метод addTag() и передав в него строковую константу TAG_OUTPUT.

```
data/WorkManagerBluromaticRepository.kt
``kt
import com.example.bluromatic.TAG_OUTPUT
...
val save = OneTimeWorkRequestBuilder<SaveImageToFileWorker>()
    .addTag(TAG_OUTPUT) // <- Добавить это
    .build()
```

Вместо идентификатора WorkManager вы используете тег для обозначения работы, потому что если пользователь размывает несколько изображений, все WorkRequests на сохранение изображения имеют один и тот же тег, но не один и тот же идентификатор.

Получение информации о работе Вы используете информацию WorkInfo из рабочего запроса SaveImageToFileWorker в логике, чтобы решить, какие композиты отображать в пользовательском интерфейсе на основе состояния BlurUiState.

Модель ViewModel получает эту информацию из переменной outputWorkInfo хранилища.

Теперь, когда вы пометили рабочий запрос SaveImageToFileWorker, вы можете выполнить следующие шаги для получения информации о нем:

В файле data/WorkManagerBluromaticRepository.kt вызовите метод workManager.getWorkInfosByTagLiveData(), чтобы заполнить переменную outputWorkInfo. В качестве параметра метода передайте константу TAG_OUTPUT.

```
data/WorkManagerBluromaticRepository.kt ``kt ... override val outputWorkInfo: Flow<WorkInfo?> =  
workManager.getWorkInfosByTagLiveData(TAG_OUTPUT) ...
```

Вызов метода getWorkInfosByTagLiveData() возвращает LiveData. LiveData - это наблюдаемый держатель данных с учетом жизненного цикла. Функция .asFlow() преобразует его в поток.

Вызовите функцию .asFlow(), чтобы преобразовать метод в поток. Вы преобразуете метод, чтобы приложение могло работать с Kotlin Flow вместо LiveData.

```
data/WorkManagerBluromaticRepository.kt  
``kt  
import androidx.lifecycle.asFlow  
...  
override val outputWorkInfo: Flow<WorkInfo?> =  
    workManager.getWorkInfosByTagLiveData(TAG_OUTPUT).asFlow()  
...
```

Цепочка вызовов функции преобразования .mapNotNull(), чтобы убедиться, что поток содержит значения. Для правила преобразования, если элемент не пуст, выберите первый элемент в коллекции. В противном случае верните нулевое значение. Функция преобразования удалит их, если они являются нулевым значением.

```
data/WorkManagerBluromaticRepository.kt
```

```
import kotlinx.coroutines.flow.mapNotNull  
...  
override val outputWorkInfo: Flow<WorkInfo?> =  
    workManager.getWorkInfosByTagLiveData(TAG_OUTPUT).asFlow().mapNotNull {  
        if (it.isNotEmpty()) it.first() else null  
    }  
...
```

Поскольку функция преобразования `.mapNotNull()` гарантирует, что значение существует, вы можете смело удалить `?` из типа `Flow`, поскольку он больше не должен быть типом с возможностью обнуления.

`data/WorkManagerBluromaticRepository.kt` ``kt ... override val outputWorkInfo: Flow = ...

Вам также нужно удалить символ `?` из интерфейса `BluromaticRepository`.

```
data/BluromaticRepository.kt
``kt
...
интерфейс BluromaticRepository {
// val outputWorkInfo: Flow<WorkInfo?>
    val outputWorkInfo: Flow<WorkInfo>
...
}
```

Информация `WorkInfo` выдается в виде потока из хранилища. Затем `ViewModel` потребляет ее.

Обновление состояния `BlurUiState` `ViewModel` использует информацию `WorkInfo`, переданную хранилищем из потока `outputWorkInfo`, чтобы установить значение переменной `blurUiState`.

Код пользовательского интерфейса использует значение переменной `blurUiState`, чтобы определить, какие композиции отображать.

Выполните следующие шаги, чтобы выполнить обновление `blurUiState`:

Заполните переменную `blurUiState` потоком `outputWorkInfo` из репозитория.

```
ui/BlurViewModel.kt ``kt // ... // УДАЛИТЬ // val blurUiState: StateFlow =
MutableStateFlow(BlurUiState.Default)
```

```
// ДОБАВИТЬ val blurUiState: StateFlow = bluromaticRepository.outputWorkInfo // ...
```

Затем вам нужно сопоставить значения в потоке с состояниями `BlurUiState` в зависимости от статуса работы.

Когда работа завершена, установите переменную `BlurUiState` в `BlurUiState.Complete(outputUri = «»)`.

Если работа отменена, установите переменную `blurUiState` в значение `BlurUiState.Default`.

В противном случае установите для переменной `blurUiState` значение `BlurUiState.Loading`.

```
ui/BlurViewModel.kt
``kt
import androidx.work.WorkInfo
import kotlinx.coroutines.flow.map
```

```
// ...

val blurUiState: StateFlow<BlurUiState> = bluromaticRepository.outputWorkInfo
    .map { info ->
        когда {
            info.state.isFinished -> {
                BlurUiState.Complete(outputUri = «»)
            }
            info.state == WorkInfo.State.CANCELLED -> {
                BlurUiState.Default
            }
            else -> BlurUiState.Loading
        }
    }

// ...
```

Так как вас интересует StateFlow, преобразуйте поток с помощью цепочки вызовов функции `.stateIn()`. Вызов функции `.stateIn()` требует трех аргументов:

В качестве первого параметра передайте `viewModelScope`, который является областью действия корутины, привязанной к `ViewModel`. В качестве второго параметра передайте `SharingStarted.WhileSubscribed(5_000)`. Этот параметр определяет, когда начинается и прекращается совместное использование. В качестве третьего параметра передайте `BlurUiState.Default`, который является начальным значением потока состояний.

```
ui/BlurViewModel.kt ``kt import kotlinx.coroutines.flow.stateIn import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.SharingStarted // ...
```

```
val blurUiState: StateFlow<BlurUiState> = bluromaticRepository.outputWorkInfo
    .map { info ->
        когда {
            info.state.isFinished -> {
                BlurUiState.Complete(outputUri = «»)
            }
            info.state == WorkInfo.State.CANCELLED -> {
                BlurUiState.Default
            }
            else -> BlurUiState.Loading
        }
    }.stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),
        initialValue = BlurUiState.Default
    )
```

```
// ...
```

ViewModel раскрывает информацию о состоянии пользовательского интерфейса в виде потока `StateFlow` через переменную `blurUiState`. Поток преобразуется из холодного `Flow` в горячий `StateFlow` путем вызова функции `stateIn()`.

Обновление пользовательского интерфейса

В файле `ui/BluromaticScreen.kt` вы получаете состояние пользовательского интерфейса из переменной `blurUiState` модели `ViewModel` и обновляете пользовательский интерфейс.

Блок `when` управляет пользовательским интерфейсом приложения. Этот блок `when` имеет ветвь для каждого из трех состояний `BlurUiState`.

UI обновляется в композите `BlurActions` внутри его композита `Row`. Выполните следующие шаги:

Удалите код `Button(onStartClick)` внутри `Row Composable` и замените его блоком `when` с аргументом `blurUiState`.

```
ui/BluromaticScreen.kt
```kt
...
 Row(
 modifier = modifier,
 horizontalArrangement = Arrangement.Center
) {
 // REMOVE
 // Button(
 // onClick = onStartClick,
 // modifier = Modifier.fillMaxWidth()
 //) {
 // Text(stringResource(R.string.start))
 // }
 // ADD
 when (blurUiState) {
 }
 }
...

```

Когда приложение открывается, оно находится в состоянии по умолчанию. В коде это состояние представлено как `BlurUiState.Default`.

Внутри блока `when` создайте ветку для этого состояния, как показано в следующем примере кода:

```
ui/BluromaticScreen.kt ```kt ... Row(модификатор = модификатор, horizontalArrangement =
Arrangement.Center) { when (blurUiState) { is BlurUiState.Default -> {} } } ...

```

Для состояния по умолчанию приложение показывает кнопку «Пуск».

Для параметра `onClick` в состоянии `BlurUiState.Default` передайте переменную

onStartClick, которая передается в композит.  
 Для параметра stringResourceId передайте идентификатор строкового ресурса R.string.start.

```
ui/BluromaticScreen.kt
``kt
...
 Row(
 модификатор = модификатор,
 horizontalArrangement = Arrangement.Center
) {
 when (blurUiState) {
 является BlurUiState.Default -> {
 Button(
 onClick = onStartClick,
 modifier = Modifier.fillMaxWidth()
) {
 Text(stringResource(R.string.start))
 }
 }
 }
 }
}
```

Когда приложение активно размывает изображение, это состояние BlurUiState.Loading. Для этого состояния приложение отображает кнопку «Отмена работы» и круговой индикатор выполнения.

Для параметра onClick кнопки в состоянии BlurUiState.Loading передайте переменную onCancelClick, которая передается в композит. Для параметра stringResourceId кнопки передайте идентификатор строкового ресурса R.string.cancel\_work.

```
ui/BluromaticScreen.kt ``kt import androidx.compose.material3.CircularProgressIndicator import
androidx.compose.material3.FilledTonalButton ... Row(модификатор = модификатор, horizontalArrangement
= Arrangement.Center) { when (blurUiState) { является BlurUiState.Default -> { Button(onStartClick) {
Text(stringResource(R.string.start)) } } is BlurUiState.Loading -> { FilledTonalButton(onCancelClick) {
Text(stringResource(R.string.cancel_work)) } CircularProgressIndicator(modifier =
Modifier.padding(dimensionResource(R.dimen.padding_small))) } } ...
```

> Примечание: Вы не используете кнопку «Отмена работы» прямо сейчас, но все равно создаете ее. Вы настроите ее на следующем шаге.

Последнее состояние, которое необходимо настроить, - это состояние BlurUiState.Complete, которое наступает после размыва и сохранения изображения. В это время приложение отображает только кнопку «Пуск».

Для параметра onClick в состоянии BlurUiState.Complete передайте переменную onStartClick.

Для параметра stringResourceId передайте идентификатор строкового ресурса R.string.start.



```

ui/BluromaticScreen.kt
``kt
...
 Row(
 модификатор = модификатор,
 horizontalArrangement = Arrangement.Center
) {
 when (blurUiState) {
 является BlurUiState.Default -> {
 Button(onStartClick) { Text(stringResource(R.string.start)) }
 }
 is BlurUiState.Loading -> {
 FilledTonalButton(onCancelClick) {
 Text(stringResource(R.string.cancel_work))
 }
 CircularProgressIndicator(modifier =
 Modifier.padding(dimensionResource(R.dimen.padding_small)))
 }
 is BlurUiState.Complete -> {
 Button(onStartClick) { Text(stringResource(R.string.start)) }
 }
 }
 }
}
...

```

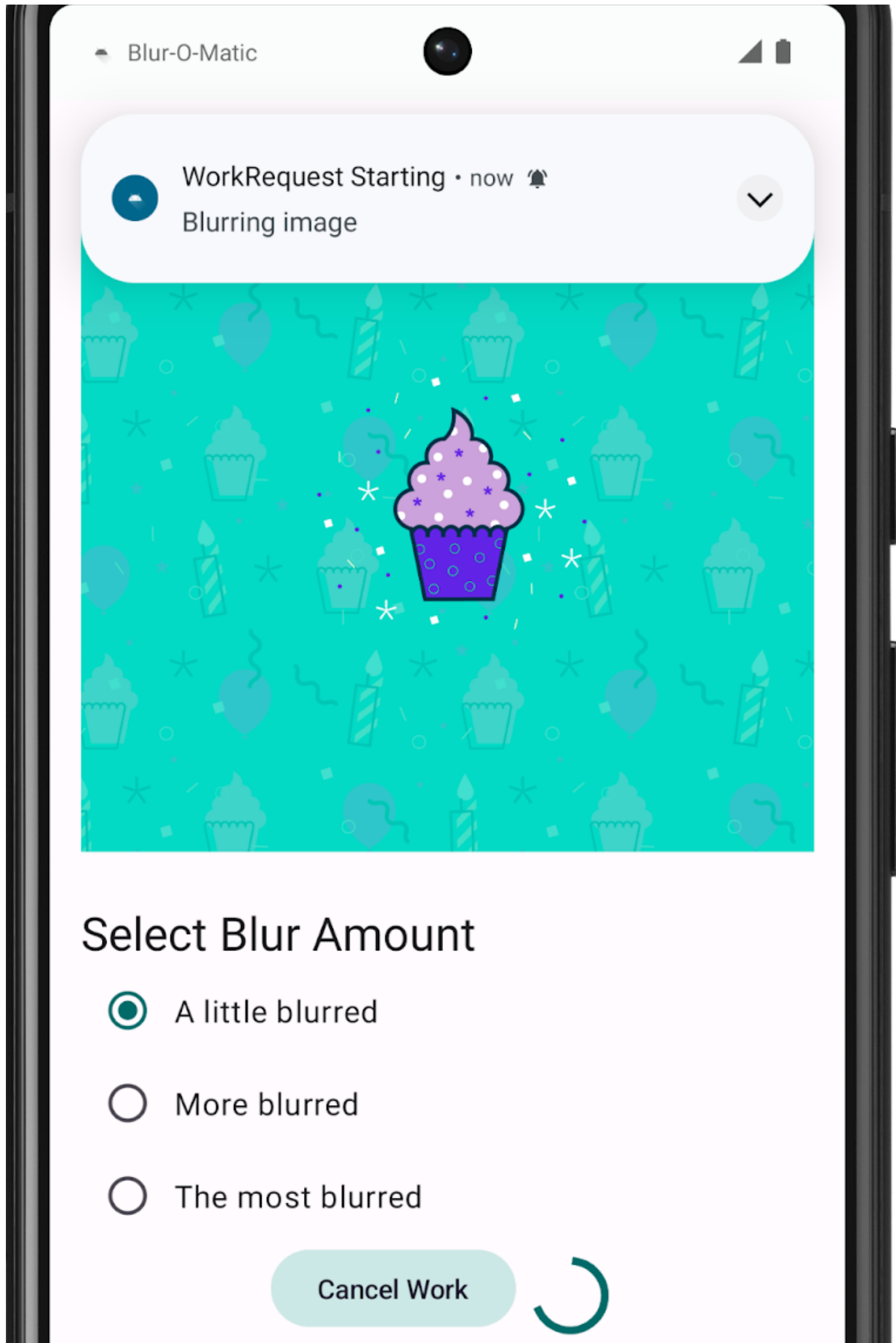
Запустите ваше приложение Запустите приложение и нажмите кнопку Start. Обратитесь к окну Background Task Inspector, чтобы увидеть, как различные состояния соответствуют отображаемому пользовательскому интерфейсу.

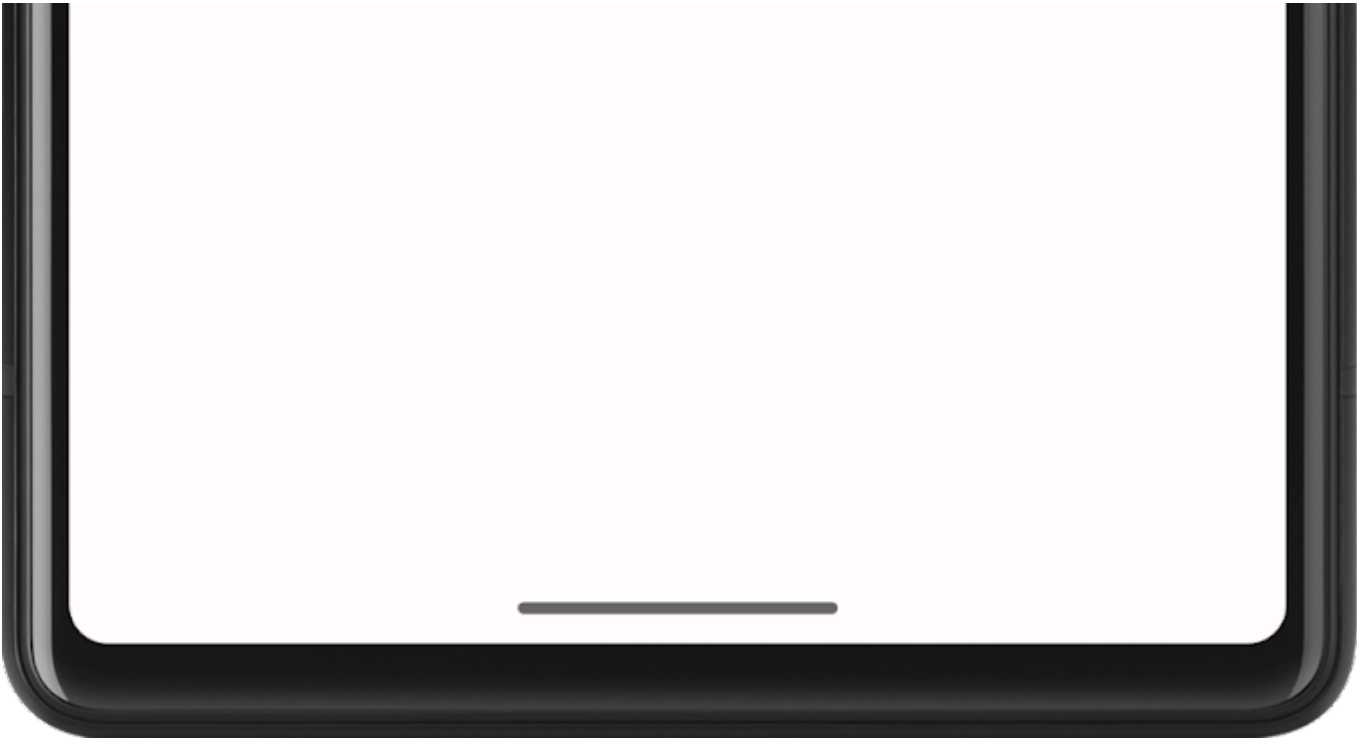
Примечание: Окно Background Task Inspector можно найти через меню: Вид > Окна инструментов > Инспекция приложений, затем выберите вкладку Инспектор фоновых задач.

SystemService - это компонент, отвечающий за управление выполнением рабочих задач.

Во время работы рабочих в пользовательском интерфейсе отображается кнопка «Отменить работу» и круговой индикатор выполнения.

App Inspection			
Pixel 6 API 33 > com.example.bluromatic			
Database Inspector Network Inspector Background Task Inspector			
All tags			
Class	Status	Start	Retries
Workers			
BlurWorker	Running	1:27:10 PM	0
SystemJobService	Started	1:27:10 PM	0
CleanupWorker	Succeeded	1:27:07 PM	0
SaveImageToFileWorker	Blocked	-	0
Jobs			
No jobs have been detected.			
Alarms			





{style="width:400px"}

После завершения работы рабочих пользовательский интерфейс обновляется и отображает кнопку «Старт», как и ожидалось.

App Inspection

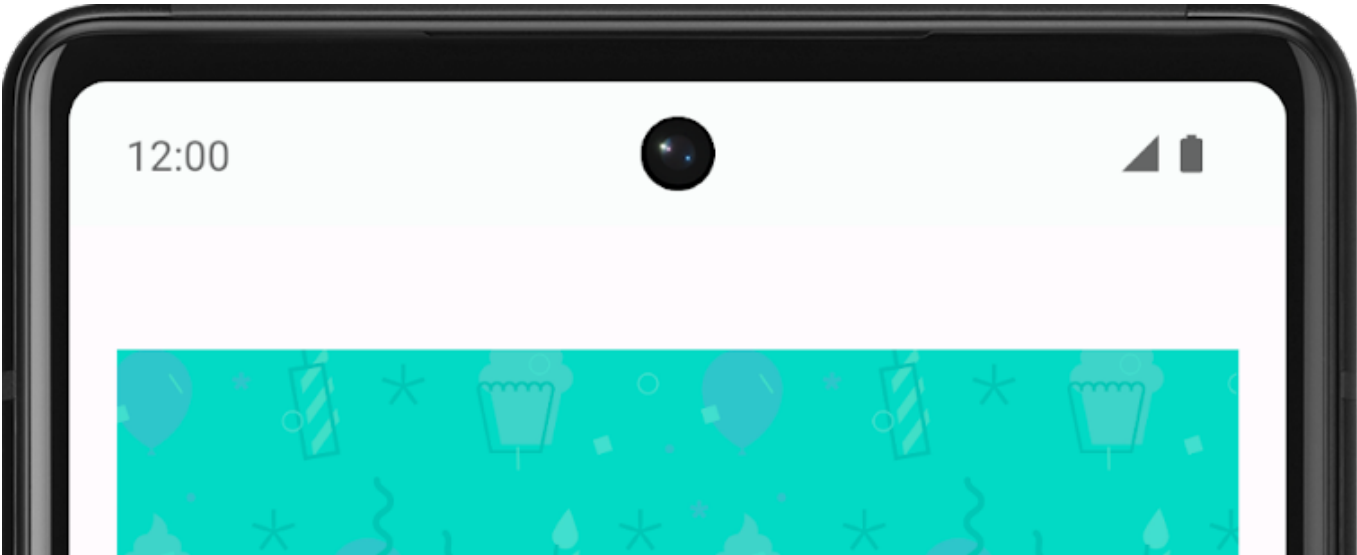
Pixel 6 API 33 > com.example.bluromatic

Database InspectorNetwork InspectorBackground Task Inspector

All tags

Class	Status	Start	Retries
Workers			
BlurWorker	✓ Succeeded	1:27:10 PM	0
CleanupWorker	✓ Succeeded	1:27:07 PM	0
SaveImageToFileWorker	✓ Succeeded	1:27:13 PM	0
Jobs			
No jobs have been detected.			-
Alarms			

GitRunProfilerLogcatApp Quality InsightsBuildTODOProblemsTerminalServicesApp Inspection





## Select Blur Amount

- ☒ A little blurred
- ☐ More blurred
- ☐ The most blurred

Start



```
{style=«width:400px»}
```

## 5. Показ окончательного вывода

В этом разделе вы настроите приложение на отображение кнопки с надписью See File всякий раз, когда размытое изображение готово к показу.

Создание кнопки See File Кнопка See File отображается только тогда, когда состояние BlurUiState равно Complete.

Откройте файл ui/BluromaticScreen.kt и перейдите к композиту BlurActions. Чтобы добавить пространство между кнопкой Start и кнопкой See File, добавьте композит Spacer внутри блока BlurUiState.Complete. Добавьте новую составную кнопку FilledTonalButton. Для параметра onClick передайте onSeeFileClick(blurUiState.outputUri). Добавьте композит Text для параметра content кнопки. Для параметра Text используйте идентификатор строкового ресурса R.string.see\_file.

```
ui/BluromaticScreen.kt ``kt import androidx.compose.foundation.layout.Spacer import
androidx.compose.foundation.layout.width
```

```
// ... is BlurUiState.Complete -> { Button(onStartClick) { Text(stringResource(R.string.start)) } // Добавляем
спейсер и новую кнопку с надписью «Посмотреть файл» Spacer(modifier =
Modifier.width(dimensionResource(R.dimen.padding_small))) FilledTonalButton({
onSeeFileClick(blurUiState.outputUri) }) { Text(stringResource(R.string.see_file)) } // ...
```

Обновление состояния BlurUiState

Состояние BlurUiState устанавливается во ViewModel и зависит от состояния рабочего запроса и, возможно, переменной bluromaticRepository.outputWorkInfo.

В файле ui/BlurViewModel.kt, внутри трансформации map(), создайте новую переменную outputImageUri.

Заполните эту новую переменную URI сохраненного изображения из объекта данных outputData.

Вы можете получить эту строку с помощью ключа KEY\_IMAGE\_URI.

```
ui/BlurViewModel.kt
``kt
import com.example.bluromatic.KEY_IMAGE_URI

// ...
.map { info ->
 val outputImageUri = info.outputData.getString(KEY_IMAGE_URI)
```

```

 when {
 // ...

```

Если рабочий завершает работу и переменная заполнена, это означает, что существует размытое изображение для отображения. Вы можете проверить, заполнена ли эта переменная, вызвав `outputImageUri.isNullOrEmpty()`.

Обновите ветвь `isFinished`, чтобы также проверить, заполнена ли переменная, а затем передайте переменную `outputImageUri` в объект данных `BlurUiState.Complete`.

```

ui/BlurViewModel.kt ``kt //map { info -> val outputImageUri = info.outputData.getString(KEY_IMAGE_URI)
когда { info.state.isFinished && !outputImageUri.isNullOrEmpty() -> { BlurUiState.Complete(outputUri =
outputImageUri) } info.state == WorkInfo.State.CANCELLED -> { // ...

```

Создание кода события нажатия кнопки See File

Когда пользователь нажимает на кнопку See File, ее обработчик `onClick` вызывает назначенную ему функцию. Эта функция передается в качестве аргумента при вызове композита `BlurActions()`.

Цель этой функции - отобразить сохраненное изображение из его URI. Она вызывает вспомогательную функцию `showBlurredImage()` и передает URI. Вспомогательная функция создает намерение и использует его для запуска новой активности, чтобы показать сохраненное изображение.

Откройте файл `ui/BluromaticScreen.kt`.

В функции `BluromaticScreenContent()`, в вызове композитной функции `BlurActions()`, начните создавать лямбда-функцию для параметра `onSeeFileClick`, которая принимает единственный параметр с именем `currentUri`. Этот подход позволяет хранить URI сохраненного изображения.

```

ui/BluromaticScreen.kt
````kt
// ...
BlurActions(
    blurUiState = blurUiState,
    onStartClick = { applyBlur(selectedValue) },
    onSeeFileClick = { currentUri ->
    },
    onCancelClick = { cancelWork() },
    modifier = Modifier.fillMaxWidth()
)
// ...

```

Внутри тела лямбда-функции вызовите вспомогательную функцию `showBlurredImage()`. В качестве первого параметра передайте переменную `context`. В качестве второго параметра передайте переменную `currentUri`.

```
ui/BluromaticScreen.kt ``kt // ... BlurActions( blurUiState = blurUiState, onStartClick = {
applyBlur(selectedValue) } // Новый лямбда-код запускается при нажатии кнопки See File onSeeFileClick =
{ currentUri -> showBlurredImage(context, currentUri) }, onCancelClick = { cancelWork() }, modifier =
Modifier.fillMaxWidth() ) // ...
```

Запустите ваше приложение

Запустите ваше приложение. Теперь вы видите новую, кликабельную кнопку See File, которая ведет вас к сохраненному файлу:

```
{style=«width:400px»}
```

```
{style=«width:400px»}
```

6. Отмена работы

```

```

Ранее вы добавили кнопку «Отменить работу», теперь вы можете добавить код, чтобы заставить ее что-то сделать. В WorkManager вы можете отменить работу, используя id, тег и уникальное имя цепочки.

В данном случае вы хотите отменить работу с помощью уникального имени цепочки, потому что вы хотите отменить всю работу в цепочке, а не только определенный шаг.

Отмена работы по имени

Откройте файл data/WorkManagerBluromaticRepository.kt.

В функции cancelWork() вызовите функцию workManager.cancelUniqueWork().

Передайте уникальное имя цепочки IMAGE_MANIPULATION_WORK_NAME, чтобы вызов отменял только запланированные работы с таким именем.

```
data/WorkManagerBluromaticRepository.kt
``kt
override fun cancelWork() {
    workManager.cancelUniqueWork(IMAGE_MANIPULATION_WORK_NAME)
}
```

Следуя принципу разделения забот, композитные функции не должны напрямую взаимодействовать с хранилищем. Композитные функции взаимодействуют с ViewModel, а ViewModel взаимодействует с хранилищем.

Такой подход является хорошим принципом проектирования, поскольку изменения в репозитории не требуют изменения композитных функций, так как они не взаимодействуют напрямую.

Откройте файл `ui/BlurViewModel.kt`. Создайте новую функцию `cancelWork()` для отмены работы. Внутри функции на объекте `bluromaticRepository` вызовите метод `cancelWork()`.

`ui/BlurViewModel.kt` ``kt /**

- Вызов метода из репозитория для отмены текущего запроса на работу (`WorkRequest`)
- `*/ fun cancelWork() { bluromaticRepository.cancelWork() }`

Настройка события клика «Отмена работы

Откройте файл `ui/BluromaticScreen.kt`.

Перейдите к композитной функции `BluromaticScreen()`.

`ui/BluromaticScreen.kt`

``kt

```
fun BluromaticScreen(blurViewModel: BlurViewModel = viewModel(factory =
BlurViewModel.Factory)) {
    val uiState by blurViewModel.blurUiState.collectAsStateWithLifecycle()
    val layoutDirection = LocalLayoutDirection.current
    Surface(
        модификатор = Модификатор
            .fillMaxSize()
            .statusBarsPadding()
            .padding(
                start = WindowInsets.safeDrawing
                    .asPaddingValues()
                    .calculateStartPadding(layoutDirection),
                end = WindowInsets.safeDrawing
                    .asPaddingValues()
                    .calculateEndPadding(layoutDirection)
            )
    ) {
        BluromaticScreenContent(
            blurUiState = uiState,
            blurAmountOptions = blurViewModel.blurAmount,
            applyBlur = blurViewModel::applyBlur,
            cancelWork = {},
            модификатор = Модификатор
                .verticalScroll(rememberScrollState())
                .padding(dimensionResource(R.dimen.padding_medium))
        )
    }
}
```

Внутри вызова композита `BluromaticScreenContent` вы хотите, чтобы метод `ViewModel cancelWork()` выполнялся, когда пользователь нажмет на кнопку.

Присвойте параметру `cancelWork` значение `blurViewModel::cancelWork`.

`ui/BluromaticScreen.kt` ``kt // ... BluromaticScreenContent(blurUiState = uiState, blurAmountOptions = blurViewModel.blurAmount, applyBlur = blurViewModel::applyBlur, cancelWork =


```
blurViewModel::cancelWork, модификатор = Модификатор.verticalScroll(rememberScrollState())
.padding(dimensionResource(R.dimen.padding_medium)) ) // ...
```

Запустите приложение и отмените работу

Запустите ваше приложение. Оно компилируется просто отлично. Начните размывать изображение, а затем нажмите «Отменить работу». Вся цепочка отменяется!

```
{style=«width:400px»}
```

После отмены работы отображается только кнопка Start, потому что `WorkInfo.State` стало `CANCELLED`. Это изменение приводит к установке переменной `blurUiState` в значение `BlurUiState.Default`, что возвращает пользовательский интерфейс в исходное состояние и отображает только кнопку Start.

В инспекторе фоновых задач отображается статус «Отменено», что вполне ожидаемо.

```

```

7. Ограничения работы

И последнее, но не менее важное: `WorkManager` поддерживает ограничения. Ограничение - это требование, которое должно быть выполнено перед запуском рабочего запроса.

Примерами ограничений являются `requiresDeviceIdle()` и `requiresStorageNotLow()`.

Для ограничения `requiresDeviceIdle()`, если ему передано значение `true`, работа будет выполняться только в том случае, если устройство простаивает.

Для ограничения `requiresStorageNotLow()`, если ему передается значение `true`, работа выполняется только в том случае, если хранилище не разряжено.

Для `Blur-O-Matic` добавляется ограничение, согласно которому уровень заряда батареи устройства не должен быть низким до запуска рабочего запроса `blurWorker`. Это ограничение означает, что ваш рабочий запрос откладывается и запускается только после того, как заряд батареи устройства не разрядится.

Создайте ограничение «Батарея не разряжена

В файле `data/WorkManagerBluromaticRepository.kt` выполните следующие действия:

Перейдите к методу `applyBlur()`.

После кода, объявляющего переменную продолжения, создайте новую переменную с именем `constraints`, которая будет содержать объект `Constraints` для создаваемого ограничения.

Создайте построитель для объекта `Constraints`, вызвав функцию `Constraints.Builder()`, и присвойте его новой переменной.

```
data/WorkManagerBluromaticRepository.kt
`kt
import androidx.work.Constraints
```

```
// ...
    override fun applyBlur(blurLevel: Int) {
        // ...

        val constraints = Constraints.Builder()
    }
// ...
```

Цепляем к вызову метод `setRequiresBatteryNotLow()` и передаем ему значение `true`, чтобы `WorkRequest` выполнялся только тогда, когда батарея устройства не разряжена.

```
data/WorkManagerBluromaticRepository.kt ``kt // ... override fun applyBlur(blurLevel: Int) { // ...
```

```
        val constraints = Constraints.Builder()
            .setRequiresBatteryNotLow(true)
```

```
// ...
```

Создайте объект путем цепочки вызовов метода `.build()`.

```
data/WorkManagerBluromaticRepository.kt
``kt
// ...
    override fun applyBlur(blurLevel: Int) {
        // ...

        val constraints = Constraints.Builder()
            .setRequiresBatteryNotLow(true)
            .build()
    }
// ...
```

Чтобы добавить объект ограничений в рабочий запрос `blurBuilder`, выполните вызов метода `.setConstraints()` и передайте в него объект ограничений.

```
data/WorkManagerBluromaticRepository.kt ``kt // ...
blurBuilder.setInputData(createInputDataForWorkRequest(blurLevel, imageUri))

blurBuilder.setConstraints(constraints) // Добавляем этот код //...
```

Тест на эмуляторе

На эмуляторе измените уровень заряда в окне `Extended Controls` на 15 % или ниже, чтобы имитировать сценарий разрядки батареи, подключение зарядного устройства к сети и состояние батареи на «Не заряжается».

```
{style="width:400px"}
```

Запустите приложение и нажмите кнопку Start, чтобы начать размывать изображение. Уровень заряда батареи эмулятора низкий, поэтому WorkManager не запускает рабочий запрос blurWorker из-за ограничений. Он ставится в очередь, но откладывается до тех пор, пока ограничение не будет выполнено. Эту отсрочку можно увидеть на вкладке Инспектор фоновых задач.

{style=«width:400px»}

После того как вы убедитесь, что задача не запущена, медленно увеличьте уровень заряда батареи.

Ограничение будет выполнено, когда уровень заряда батареи достигнет примерно 25 %, и отложенная работа будет запущена. Этот результат отображается на вкладке Инспектор фоновых задач.

{style=«width:400px»}

> Примечание: Еще одним хорошим ограничением, которое можно добавить в `Blur-O-Matic`, является ограничение `setRequiresStorageNotLow()` при сохранении. Полный список возможностей ограничений можно найти в справочнике `Constraints.Builder`.

8. Написание тестов для реализаций Worker

Как тестировать WorkManager

Написание тестов для Worker'ов и тестирование с использованием API WorkManager может быть нелогичным. Работа, выполняемая в Worker, не имеет прямого доступа к пользовательскому интерфейсу - это сугубо бизнес-логика. Обычно бизнес-логику тестируют с помощью локальных модульных тестов. Однако вы можете помнить из урока Background Work with WorkManager, что для работы WorkManger требуется Android Context. Контекст по умолчанию недоступен в локальных модульных тестах. Поэтому вы должны тестировать тесты Worker вместе с UI-тестами, даже несмотря на отсутствие прямых элементов UI, которые нужно тестировать.

Настройка зависимостей

Вам нужно добавить три зависимости gradle в ваш проект. Первые две включают JUnit и espresso для UI-тестов. Третья зависимость предоставляет API для тестирования работы.

```
app/build.gradle.kts
```

```
зависимости { // Espresso androidTestImplementation(«androidx.test.espresso:espresso-core:3.5.1») // Junit
androidTestImplementation(«androidx.test.ext:junit:1.1.5») // Рабочее тестирование
androidTestImplementation(«androidx.work:work-testing:2.8.1») }
```

В своем приложении вы должны использовать самую актуальную стабильную версию work-runtime-ktx. Если вы измените версию, не забудьте нажать Sync Now, чтобы синхронизировать проект с обновленными gradle-файлами.

Создайте класс тестов

Создайте каталог для UI-тестов в каталоге `app > src`.

Создайте новый Kotlin-класс в каталоге `androidTest/java` под названием `WorkerInstrumentationTest`.

Напишите тест `CleanupWorker`

Выполните следующие шаги, чтобы написать тест для проверки реализации `CleanupWorker`. Попробуйте выполнить эту проверку самостоятельно, основываясь на инструкциях. Решение приведено в конце шагов.

В файле `WorkerInstrumentationTest.kt` создайте переменную `lateinit` для хранения экземпляра `Context`.

Создайте метод `setUp()`, аннотированный `@Before`.

В методе `setUp()` инициализируйте переменную `lateinit context` контекстом приложения из `ApplicationProvider`.

Создайте тестовую функцию `cleanupWorker_doWork_resultSuccess()`.

В тесте `cleanupWorker_doWork_resultSuccess()` создайте экземпляр `CleanupWorker`.

```
WorkerInstrumentationTest.kt
``kt
class WorkerInstrumentationTest {
    private lateinit var context: Context

    @Before
    fun setUp() {
        context = ApplicationProvider.getApplicationContext()
    }

    @Test
    fun cleanupWorker_doWork_resultSuccess() {
    }
}
```

При написании приложения **Blur-O-Matic** вы используете `OneTimeWorkRequestBuilder` для создания рабочих. Для тестирования рабочих требуются разные конструкторы. API `WorkManager` предоставляет два различных конструктора:

`TestWorkerBuilder` `TestListenableWorkerBuilder`. Оба этих конструктора позволяют протестировать бизнес-логику вашего рабочего. Для таких `CoroutineWorker`, как `CleanupWorker`, `BlurWorker` и `SaveImageToFileWorker`, используйте `TestListenableWorkerBuilder`, потому что он обрабатывает потоковые сложности `Coroutine`.

```
// ... blurBuilder.setInputData(createInputDataForWorkRequest(blurLevel, imageUri))
blurBuilder.setConstraints(constraints) // Добавляем этот код
```

WorkerInstrumentationTest.kt

```

class WorkerInstrumentationTest {
    private lateinit var context: Context

    @Before
    fun setUp() {
        context = ApplicationProvider.getApplicationContext()
    }

    @Test
    fun cleanupWorker_doWork_resultSuccess() {
        val worker = TestListenableWorkerBuilder<CleanupWorker>(context).build()
        runBlocking {
        }
    }
}

```

В теле лямбды `runBlocking` вызовите `doWork()` для экземпляра `CleanupWorker`, который вы создали в шаге 5, и сохраните его как значение. Возможно, вы помните, что `CleanupWorker` удаляет все файлы PNG, сохраненные в файловой структуре приложения **Blur-O-Matic**. Этот процесс включает в себя ввод/вывод файлов, что означает, что при попытке удаления файлов могут возникнуть исключения. По этой причине попытка удаления файлов обернута в блок `try`.

```

CleanupWorker.kt ``kt ... return@withContext try { val outputDirectory = File(applicationContext.filesDir,
OUTPUT_PATH) if (outputDirectory.exists()) { val entries = outputDirectory.listFiles() if (entries != null) { for
(entry in entries) { val name = entry.name if (name.isNotEmpty() && name.endsWith(«.png»)) { val deleted =
entry.delete() Log.i(TAG, «Удалено $name - $deleted») } } } Result.success() } catch (exception: Exception) {
Log.e( TAG, applicationContext.resources.getString(R.string.error_cleaning_file), исключение ) Result.failure() }

```

Обратите внимание, что в конце блока `try` возвращается `Result.success()`. Если код дошел до `Result.success()`, то ошибки доступа к каталогу файлов нет.

Теперь пришло время сделать утверждение, которое указывает на то, что рабочий был успешным.

Утверждаем, что результатом работы рабочего является `ListenableWorker.Result.success()`.
Взгляните на следующий код решения:

```

WorkerInstrumentationTest.kt
``kt
class WorkerInstrumentationTest {
    private lateinit var context: Context

    @Before
    fun setUp() {
        context = ApplicationProvider.getApplicationContext()
    }
}

```

```

    }

    @Test
    fun cleanupWorker_doWork_resultSuccess() {
        val worker = TestListenableWorkerBuilder<CleanupWorker>(context).build()
        runBlocking {
            val result = worker.doWork()
            assertTrue(result is ListenableWorker.Result.Success)
        }
    }
}

```

Напишите тест BlurWorker @Before fun setUp() { context = ApplicationProvider.getApplicationContext()

} @Test fun cleanupWorker_doWork_resultSuccess() {

}} ``

При написании приложения **Blur-O-Matic** вы используете OneTimeWorkRequestBuilder для создания рабочих. Для тестирования рабочих требуются разные конструкторы. API WorkManager предоставляет два различных конструктора:

Создайте блок runBlocking. Вызовите doWork() внутри блока runBlocking. В отличие от CleanupWorker, BlurWorker имеет некоторые выходные данные, которые можно протестировать!

Чтобы получить доступ к выходным данным, извлеките URI из результата doWork().

WorkerInstrumentationTest.kt ``kt @Test fun blurWorker_doWork_resultSuccessReturnsUri() { val worker = TestListenableWorkerBuilder(context) .setInputData(workDataOf(mockUriInput)) .build() runBlocking { val result = worker.doWork() val resultUri = result.outputData.getString(KEY_IMAGE_URI) } }

Сделайте утверждение об успешной работе рабочего. Для примера посмотрите на следующий код из BlurWorker:

```

BlurWorker.kt
``kt
val resourceUri = inputData.getString(KEY_IMAGE_URI)
val BlurLevel = inputData.getInt(BLUR_LEVEL, 1)

...
val picture = BitmapFactory.decodeStream(
    resolver.openInputStream(Uri.parse(resourceUri))
)

val output = blurBitmap(picture, blurLevel)

// Запись растрового изображения в временный файл
val outputUri = writeBitmapToFile(applicationContext, output)

val outputData = workDataOf(KEY_IMAGE_URI to outputUri.toString())

```

```
Result.success(outputData)
...
```

BlurWorker берет URI и уровень размытия из входных данных и создает временный файл. Если операция прошла успешно, он возвращает пару ключ-значение, содержащую URI. Чтобы проверить правильность содержимого выходных данных, сделайте утверждение, что выходные данные содержат ключ KEY_IMAGE_URI.

Сделайте утверждение, что выходные данные содержат URI, начинающийся со строки "file:///data/user/0/com.example.bluromatic/files/blur_filter_outputs/blur-filter-output-"

Примечание: URI из результата doWork() может быть null. Если URI равен null, верните false внутри утверждения.

Проверьте свой тест на примере следующего кода решения:

WorkerInstrumentationTest.kt

```
@Test
fun blurWorker_doWork_resultSuccessReturnsUri() {
    val worker = TestListenableWorkerBuilder<BlurWorker>(context)
        .setInputData(workDataOf(mockUriInput))
        .build()
    runBlocking {
        val result = worker.doWork()
        val resultUri = result.outputData.getString(KEY_IMAGE_URI)
        assertTrue(result is ListenableWorker.Result.Success)
        assertTrue(result.outputData.keyValueMap.containsKey(KEY_IMAGE_URI))
        assertTrue(
            resultUri?.startsWith("file:///data/user/0/com.example.bluromatic/files/blur_filter_outputs/blur-filter-output-")
                ?: false
        )
    }
}
```

Напишите тест SaveImageToFileWorker в соответствии со своим названием, SaveImageToFileWorker записывает файл на диск. Напомним, что в WorkManagerBluromaticRepository вы добавляете SaveImageToFileWorker в WorkManager в качестве продолжения после BlurWorker. Поэтому у него те же входные данные. Он получает URI из входных данных, создает растровое изображение, а затем записывает его на диск в виде файла. Если операция выполнена успешно, то результатом будет URL-адрес изображения. Тест для SaveImageToFileWorker очень похож на тест для BlurWorker, единственное отличие - выходные данные.

Попробуйте самостоятельно написать тест для SaveImageToFileWorker! Когда вы закончите, можете проверить решение, приведенное ниже. Вспомните подход, который вы использовали для теста BlurWorker:

Создайте рабочий, передав ему входные данные. Создайте блок `runBlocking`. Вызовите `doWork()` на рабочем. Проверьте, что результат был успешным. Проверьте правильность ключа и значения в выходных данных.

Примечание: URL-адрес файла изображения, который рабочий `SaveImageToFileWorker` сохраняет на диск, должен начинаться со следующей строки: «`content://media/external/images/media/`».

Вот решение:

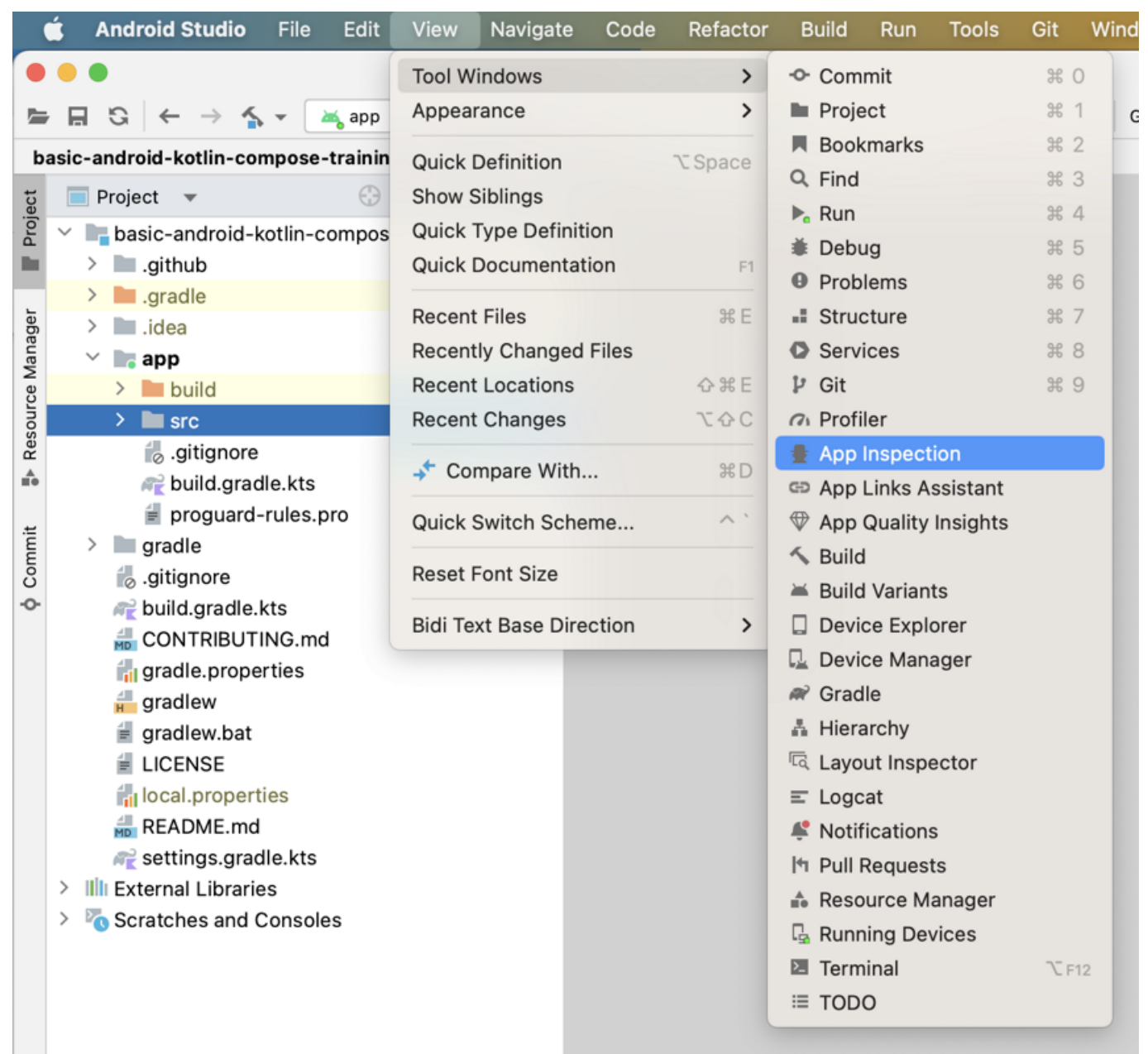
```
@Test
fun saveImageToFileWorker_doWork_resultSuccessReturnsUrl() {
    val worker = TestListenableWorkerBuilder<SaveImageToFileWorker>(context)
        .setInputData(workDataOf(mockUriInput))
        .build()
    runBlocking {
        val result = worker.doWork()
        val resultUri = result.outputData.getString(KEY_IMAGE_URI)
        assertTrue(result is ListenableWorker.Result.Success)
        assertTrue(result.outputData.keyValueMap.containsKey(KEY_IMAGE_URI))
        assertTrue(
            resultUri?.startsWith("content://media/external/images/media/")
                ?: false
        )
    }
}
```

9. Отладка WorkManager с помощью инспектора фоновых задач

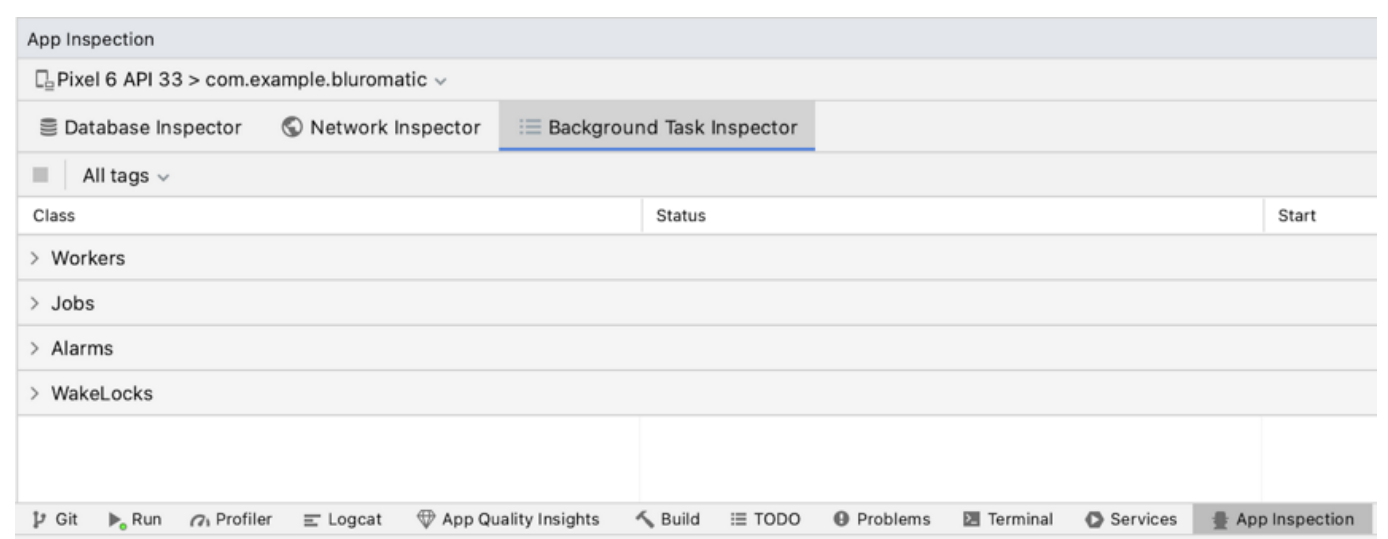
Проверка рабочих Автоматические тесты - отличный способ проверить функциональность ваших Workers. Однако они не так полезны, когда вы пытаетесь отладить Worker. К счастью, в Android Studio есть инструмент, позволяющий визуализировать, отслеживать и отлаживать Workers в режиме реального времени. Фоновый инспектор задач работает на эмуляторах и устройствах с уровнем API 26 и выше.

В этом разделе вы узнаете о некоторых функциях Background Task Inspector для проверки рабочих в **Blur-O-Matic**.

Запустите приложение **Blur-O-Matic** на устройстве или эмуляторе. Перейдите в меню Вид > Окна инструментов > Инспекция приложений.

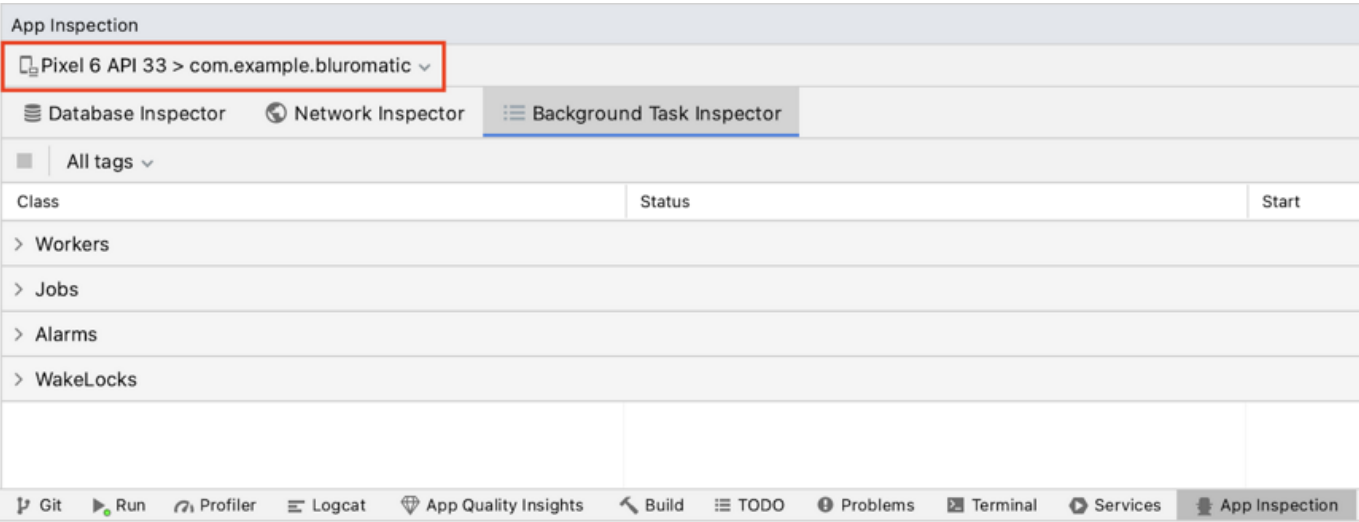


Выберите вкладку Инспектор фоновых задач.

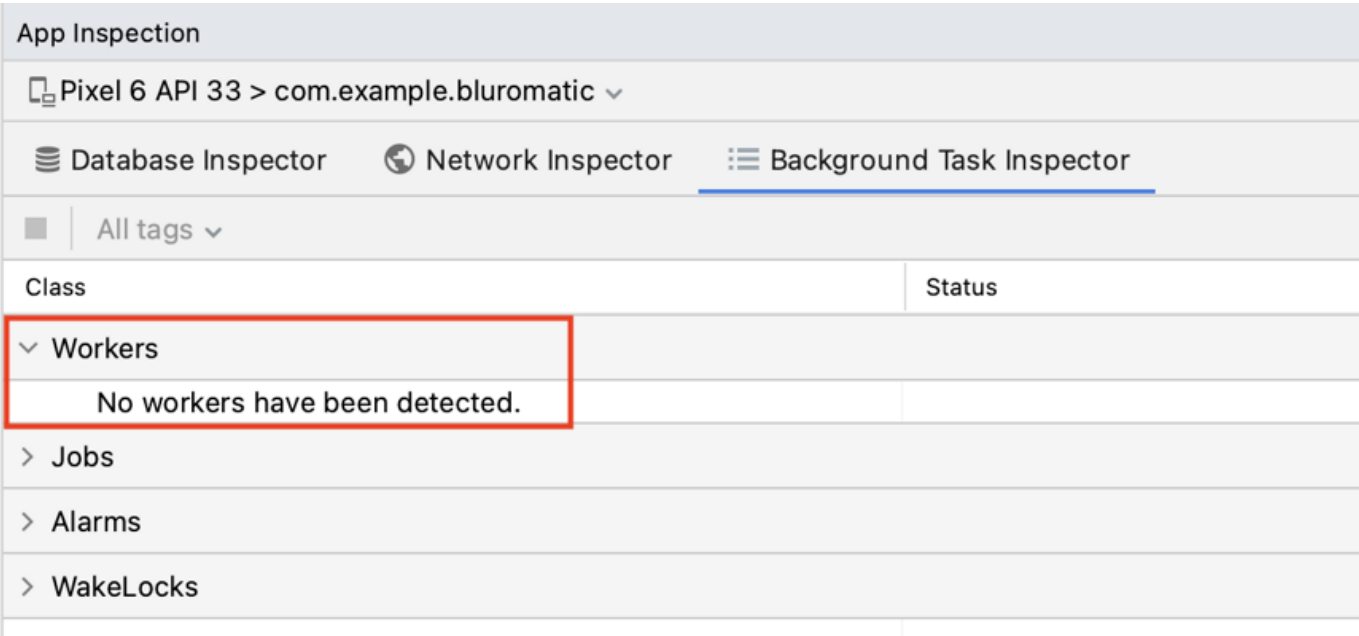


При необходимости выберите устройство и запущенный процесс из выпадающего меню. В примерах изображений процесс - com.example.bluromatic. Возможно, процесс будет выбран автоматически. Если

он выберет неправильный процесс, вы можете изменить его.



Щелкните раскрывающееся меню «Рабочие». В настоящее время рабочих нет, что вполне логично, поскольку попыток размыть изображение не было.



В приложении выберите More blurred и нажмите Start. Вы сразу же увидите некоторое содержимое в раскрывающемся списке Рабочие.

Примечание: Помните, что инспектор фоновых задач не похож на отладчик в том смысле, что он не останавливает рабочие процессы во время их выполнения; отладчик предоставляет эту функцию отдельно. Фоновый инспектор задач только следит за рабочими во время их выполнения.

Теперь в раскрывающемся списке Workers вы видите примерно следующее.

Workers	
SaveltImageToFileWorker	✓ Succeeded
SystemJobService	⌛ Stopped
CleanupWorker	✓ Succeeded
BlurWorker	✓ Succeeded
SystemJobService	⌛ Scheduled

В таблице Worker отображается имя рабочего, служба (в данном случае SystemJobService), статус каждого из них и временная метка. На скриншоте из предыдущего шага видно, что BlurWorker и CleanupWorker успешно завершили свою работу.

Вы также можете отменить работу с помощью инспектора.

Выберите работника, поставленного в очередь, и нажмите кнопку Отменить выбранного работника 7108c2a82f64b348.png на панели инструментов. Просмотр сведений о задании Щелкните рабочего в таблице Рабочие.

App Inspection			
Pixel 6 API 33 > com.example.bluromatic			
Database Inspector Network Inspector Background Task Inspector			
All tags			
Class	Status	Start	Retries
Workers			
SaveltImageToFileWorker	✓ Succeeded	4:42:22 PM	0
SystemJobService	⌛ Stopped	4:42:22 PM	0
CleanupWorker	✓ Succeeded	4:42:14 PM	0
BlurWorker	✓ Succeeded	4:42:17 PM	0
SystemJobService	⌛ Scheduled	4:42:17 PM	0

При этом откроется окно Сведения о задаче.

App Inspection

Pixel 6 API 33 > com.example.bluromatic

Database Inspector

Network Inspector

Background Task Inspector

All tags

Class	Status	Start	Retries
Workers			
SaveltImageToFileWork	✓ Succeeded	4:42:22 PM	0
SystemJobService	⌛ Stopped	4:42:22 PM	0
CleanupWorker	✓ Succeeded	4:42:14 PM	0
BlurWorker	✓ Succeeded	4:42:17 PM	0
SystemJobService	⌛ Scheduled	4:42:17 PM	0
Jobs			
No jobs have been de			
Alarms			
WakeLocks			

Task Details

Description

Class
com.example.bluromatic.workers.BlurWorker

Tags
"com.example.bluromatic.workers.BlurWorker"

UUID
a0273452-626e-4a06-9d1e-78d6d52dad0e

Execution

Enqueued by
WorkManagerBluromaticRepository.kt (85)

Constraints
Requires battery not low

Frequency
OneTime

State
✓ Succeeded

Related Job
⌛ SystemJobService

WorkContinuation

Show in graph

Previous
✓ 7320a2f2-4c39-4c2f-87f1-881268860b3e

Next
✓ 227b4897-2ad3-4281-9579-3d6449c1a623

Unique work chain
✓ 7320a2f2-4c39-4c2f-87f1-881268860b3e
✓ a0273452-626e-4a06-9d1e-78d6d52dad0e (Current)
✓ 227b4897-2ad3-4281-9579-3d6449c1a623

Results

Time started
4:42:17 PM

Просмотрите информацию, отображенную в разделе «Сведения о задании».

Task Details	
Description	
Class	com.example.bluromatic.workers.BlurWorker
Tags	"com.example.bluromatic.workers.BlurWorker"
UUID	a0273452-626e-4a06-9d1e-78d6d52dad0e
Execution	
Enqueued by	WorkManagerBluromaticRepository.kt (85)
Constraints	Requires battery not low
Frequency	OneTime
State	✔ Succeeded
Related Job	🕒 SystemJobService
WorkContinuation	
Show in graph	
Previous	✔ 7320a2f2-4c39-4c2f-87f1-881268860b3e
Next	✔ 227b4897-2ad3-4281-9579-3d6449c1a623
Unique work chain	✔ 7320a2f2-4c39-4c2f-87f1-881268860b3e
	✔ a0273452-626e-4a06-9d1e-78d6d52dad0e (Current)
	✔ 227b4897-2ad3-4281-9579-3d6449c1a623
Results	
Time started	4:42:17 PM
Retries	1
Output data	▼ Data KEY_IMAGE_URI = "file:///data/user/0/com.example.bluromatic/files/blur_filter_outputs/blur-filter-output-b5585fd2-5400-4286-b464-6adb674cfb99.png"

В деталях показаны следующие категории:

Описание: В этом разделе перечислено имя класса рабочего с полностью определенным пакетом, а также назначенный тег и UUID этого рабочего. Выполнение: В этом разделе отображаются ограничения рабочего (если таковые имеются), частота выполнения, его состояние, а также то, какой класс создал и поставил в очередь этот рабочий. Вспомните, что у BlurWorker есть ограничение, которое не позволяет ему выполняться, когда батарея разряжена. Когда вы осматриваете рабочий, у которого есть ограничения, они появляются в этом разделе. WorkContinuation: В этом разделе отображается место данного рабочего в рабочей цепочке. Чтобы узнать подробности о другом работнике в рабочей цепочке, щелкните на его UUID. Результаты: В этом разделе отображается время начала, количество повторных попыток и выходные данные выбранного рабочего.

Вид графика Напомним, что рабочие в **Blur-O-Matic** связаны между собой цепочкой. Инспектор фоновых задач предлагает графическое представление, которое наглядно отображает зависимости между рабочими.

В углу окна Background Task Inspector есть две кнопки для переключения между режимами - Show Graph View и Show List View.

App Inspection

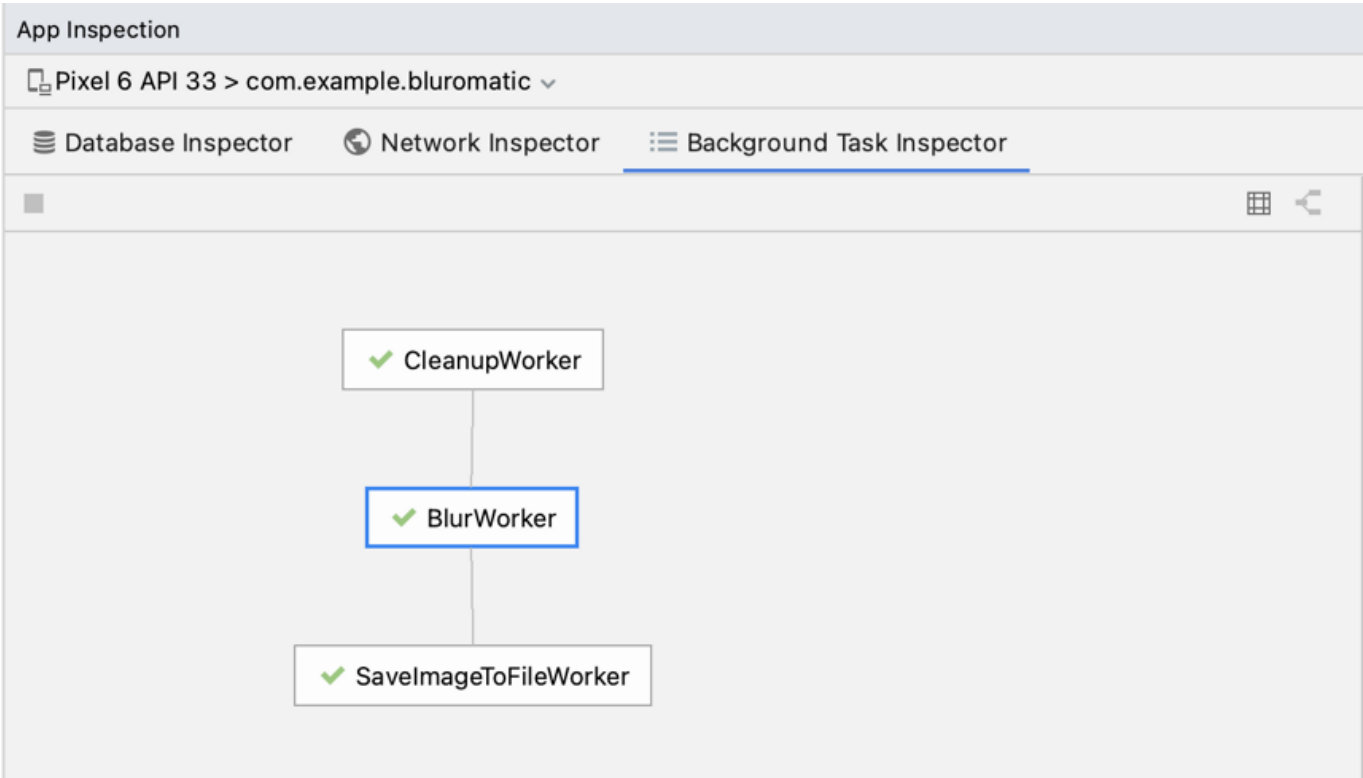
Pixel 6 API 33 > com.example.bluromatic

Database InspectorNetwork InspectorBackground Task Inspector

All tags

Class	Status	Start	Retries
Workers			
SaveImageToFileWorker	Succeeded	4:42:22 PM	0
SystemJobService	Stopped	4:42:22 PM	0
CleanupWorker	Succeeded	4:42:14 PM	0
BlurWorker	Succeeded	4:42:17 PM	0
SystemJobService	Scheduled	4:42:17 PM	0

Нажмите кнопку Показать вид графика :



Вид графика точно отображает зависимость Worker, реализованную в приложении **Blur-O-Matic**.

Нажмите кнопку Показать представление списка чтобы выйти из режима просмотра графика. Дополнительные возможности В приложении **Blur-O-Matic** реализованы только Рабочие для выполнения фоновых задач. Однако вы можете прочитать больше об инструментах, доступных для проверки других типов фоновой работы, в документации к инспектору фоновых задач.

Получение кода решения

Чтобы загрузить код готового урока, вы можете воспользоваться следующими командами:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-workmanager.git
$ cd basic-android-kotlin-compose-training-workmanager
$ git checkout main
```

Заключение

Вы узнали о дополнительной функциональности WorkManger, написали автоматические тесты для рабочих **Blur-O-Matic** и использовали инспектор фоновых задач для их изучения.

В этом уроке вы узнали:

- Именовывать уникальные цепочки WorkRequest.
- Присваивать теги WorkRequests.
- Обновление пользовательского интерфейса на основе информации о работе.
- Отмена WorkRequest.
- Добавление ограничений в WorkRequest.
- API тестирования WorkManager.
- Как подходить к тестированию реализаций рабочих.
- Как тестировать CoroutineWorkers.
- Как вручную проверить рабочие и убедиться в их функциональности.